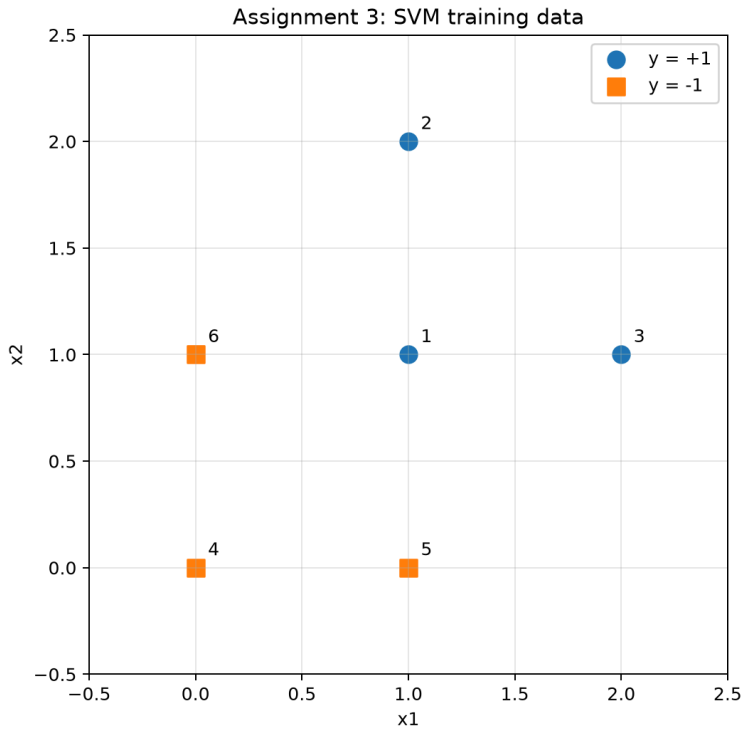
CSE 242 — Assignment 3: SVMs, K-means, and K-NN

Daniel Rothman, dmrothma@ucsc.edu

Also see my handwritten answers for math details.

Question 1: Hard-margin linear SVM



Q1(a)

The support vectors are **Point 1: (1,1)** from the positive class and **Point 5: (1,0)** and **Point 6: (0,1)** from the negative class. These are the closest points to the separating boundary and therefore determine the maximum-margin separator.

Q1(b)

The negative support vectors lie on the line `x1 + x2 = 1`, and the positive support vector lies on `x1 + x2 = 2`. Therefore, the maximum-margin decision boundary is halfway between them:

```
None
x1 + x2 = 1.5
```

An equivalent canonical SVM form is:

```
None
2x1 + 2x2 - 3 = 0
```

with `w = (2,2)` and `b = -3`.
The geometric margin is:

```
None
1 / ||w|| = 1 / (2sqrt(2)) ≈ 0.3536
```

The full width between the supporting hyperplanes is:

```
None
2 / ||w|| = 1 / sqrt(2) ≈ 0.7071
```

——————————

```
# TODO: Inspect the plot and identify all support vectors.
    # Point 1 is the closest positive example.
    # Points 5 and 6 are the closest negative examples.
    # These three points determine the maximum-margin boundary.

    return [1, 5, 6]
```

```
# TODO: Determine the geometric margin and state its meaning.
    # Positive support vector (1, 1) lies on x1 + x2 = 2.
    # Negative support vectors (1, 0) and (0, 1)
    # lie on x1 + x2 = 1.
    #
    # The maximum-margin boundary is halfway between:
    # x1 + x2 = 1.5
    #
    # Canonical SVM form:
    # 2*x1 + 2*x2 - 3 = 0

    weights = (2.0, 2.0)
    bias = -3.0

    # Geometric margin = 1 / ||w||
```

```
    # ||w|| = sqrt(w1^2 + w2^2)

    geometric_margin = 1.0 / (
        weights[0] ** 2 + weights[1] ** 2
    ) ** 0.5

    return weights, bias, geometric_margin
```

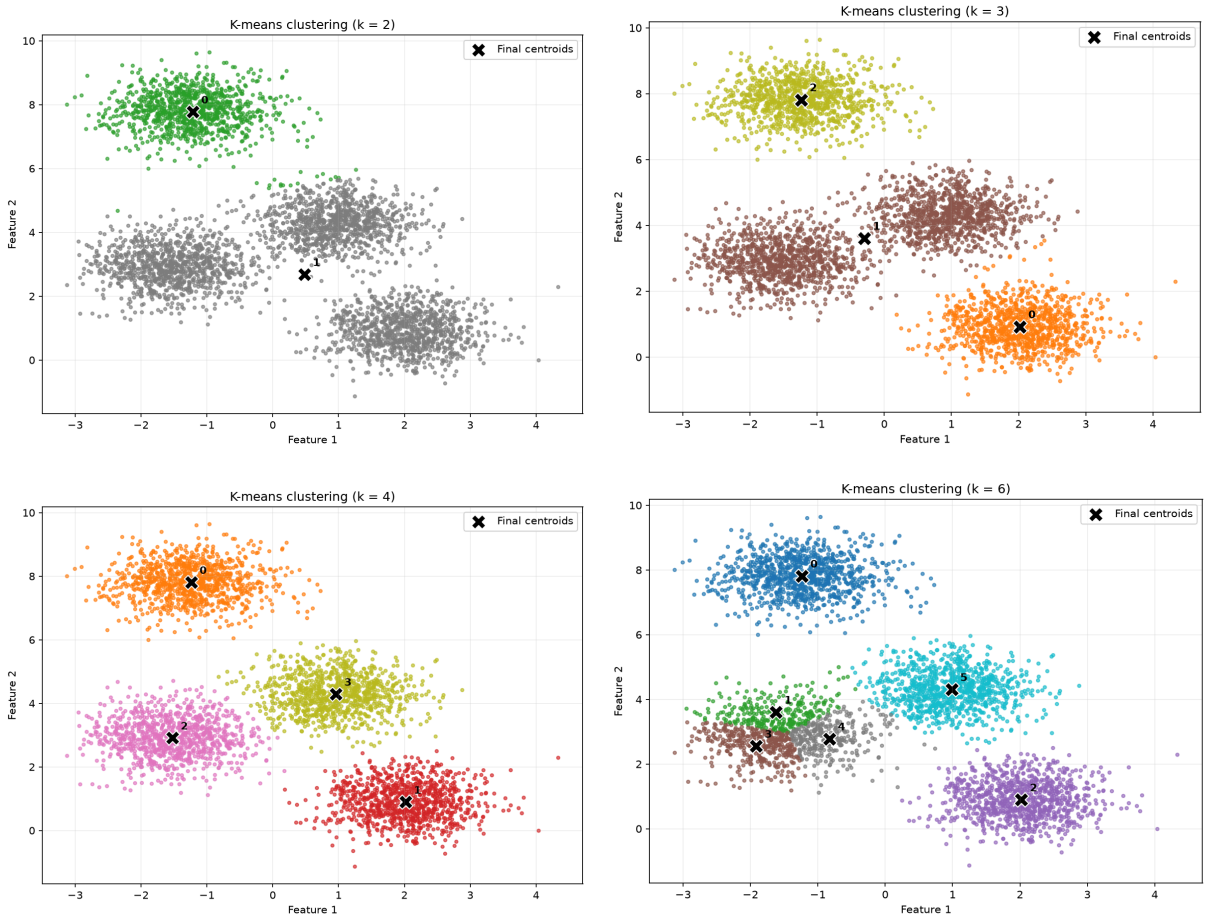
Terminal Results

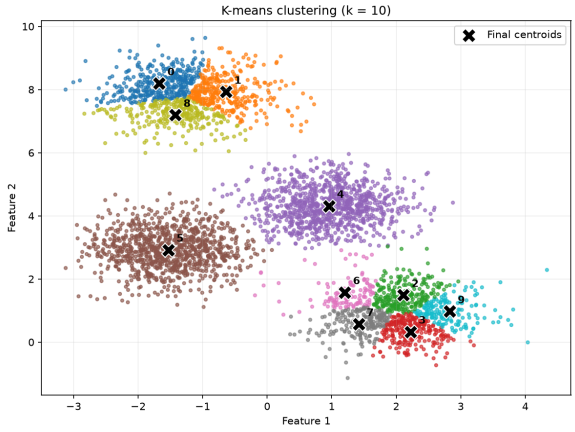
PS C:\Users\danie\Documents\CSE242\repo_assignments\Assignment3> python assignment3_template.py --task svm
Support vector point IDs: [1, 5, 6]
Weights: (2.0, 2.0); bias: -3.0; geometric margin: 0.35355339059327373

Question 2: K-means

The K-means implementation uses squared Euclidean distance to assign each observation to its nearest centroid. When distances are equal, the centroid with the lowest index is selected. Each centroid is then updated to the mean of its assigned observations. If a cluster becomes empty, its previous centroid is retained. The algorithm performs 100 iterations, followed by a final assignment step.

K-means was applied to a dataset of 4,000 points generated from four Gaussian groups, using $k = 2, 3, 4, 6$, and 10. With $k = 2$ and $k = 3$, the algorithm merged some of the original groups because too few centroids were available. At $k = 4$, the resulting clusters closely matched the four underlying groups. Increasing k to 6 and 10 subdivided some groups into smaller clusters. Although the within-cluster sum of squared errors decreased as k increased, this did not necessarily produce more meaningful clusters. These results shows the importance of selecting an appropriate number of clusters.





Terminal Results

PS C:\Users\danie\Documents\CSE242\repo_assignments\Assignment3> python kmeans_template.py --seed 0 --max-iterations 100
k= 2 | cluster sizes=[1017, 2983] | within-cluster SSE=15222.21 | saved C:\Users\danie\Documents\CSE242\repo_assignments\Assignment3\results\kmeans\kmeans_k2.png
k= 3 | cluster sizes=[1008, 1992, 1000] | within-cluster SSE=6826.13 | saved C:\Users\danie\Documents\CSE242\repo_assignments\Assignment3\results\kmeans\kmeans_k3.png
k= 4 | cluster sizes=[1000, 998, 1002, 1000] | within-cluster SSE=2794.24 | saved C:\Users\danie\Documents\CSE242\repo_assignments\Assignment3\results\kmeans\kmeans_k4.png
k= 6 | cluster sizes=[1000, 299, 997, 407, 319, 978] | within-cluster SSE=2396.79 | saved C:\Users\danie\Documents\CSE242\repo_assignments\Assignment3\results\kmeans\kmeans_k6.png
k=10 | cluster sizes=[365, 350, 253, 261, 988, 997, 123, 213, 285, 165] | within-cluster SSE=1914.79 | saved C:\Users\danie\Documents\CSE242\repo_assignments\Assignment3\results\kmeans\kmeans_k10.png

Question 3: Noisy-label SVM and K-NN repair

Question 3(a) — Noisy-label SVM baseline

The experiment used 2,000 training samples and an independently generated clean test set of 2,000 samples. Positive training labels were flipped with probability 0.35, while negative labels were flipped with probability 0.20. An RBF SVM with $C = 1$ and $\gamma = 0.01$ was trained across 20 independently corrupted training-label trials. Using random seed 0, the mean baseline test accuracy was 82.80%.

Question 3(b) — K-NN label repair

A five-nearest-neighbor majority-vote method was used to repair the corrupted training labels before SVM training. Each training point's five nearest other points were selected using Euclidean distance, and its label was replaced by their majority vote. This increased mean test accuracy to 86.44%, an improvement of 3.64 percentage points over the baseline.

K-NN repair likely helped because neighboring data points often belong to the same underlying class, allowing majority voting to reduce independent label noise. However, repair can also introduce errors near class boundaries or in overlapping distributions, where nearby points may genuinely belong to different classes. Therefore, improvement is not guaranteed for every dataset.

Baseline SVM

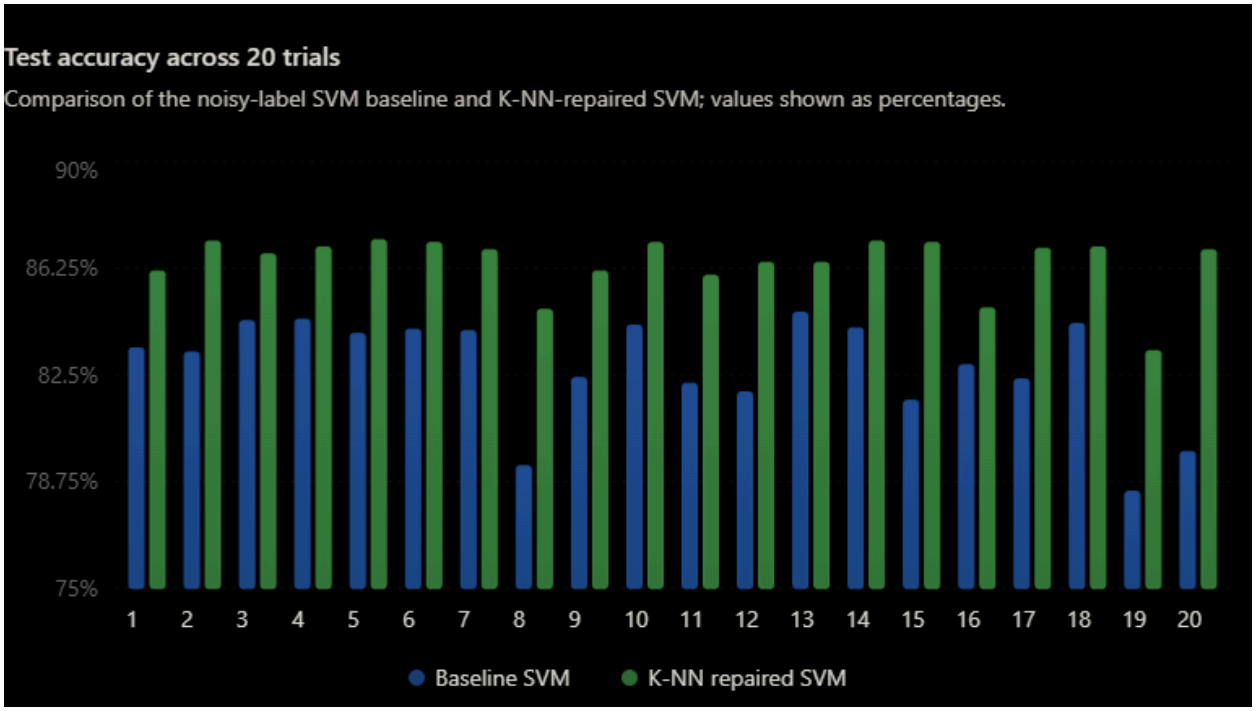
82.80%

K-NN repaired SVM

86.44%

Improvement

+3.64 pp



Terminal Results

PS C:\Users\danie\Documents\CSE242\repo_assignments\Assignment3> python noisy_classification_template.py --repeats 20 --seed 0
Trial 1: baseline=0.8345, K-NN repair=0.8615
Trial 2: baseline=0.8330, K-NN repair=0.8720
Trial 3: baseline=0.8440, K-NN repair=0.8675
Trial 4: baseline=0.8445, K-NN repair=0.8700
Trial 5: baseline=0.8395, K-NN repair=0.8725
Trial 6: baseline=0.8410, K-NN repair=0.8715
Trial 7: baseline=0.8405, K-NN repair=0.8690
Trial 8: baseline=0.7930, K-NN repair=0.8480
Trial 9: baseline=0.8240, K-NN repair=0.8615
Trial 10: baseline=0.8425, K-NN repair=0.8715
Trial 11: baseline=0.8220, K-NN repair=0.8600
Trial 12: baseline=0.8190, K-NN repair=0.8645
Trial 13: baseline=0.8470, K-NN repair=0.8645
Trial 14: baseline=0.8415, K-NN repair=0.8720
Trial 15: baseline=0.8160, K-NN repair=0.8715
Trial 16: baseline=0.8285, K-NN repair=0.8485
Trial 17: baseline=0.8235, K-NN repair=0.8695
Trial 18: baseline=0.8430, K-NN repair=0.8700
Trial 19: baseline=0.7840, K-NN repair=0.8335
Trial 20: baseline=0.7980, K-NN repair=0.8690

Baseline mean accuracy: 0.8280 (82.80%)
K-NN mean accuracy: 0.8644 (86.44%)
Change from baseline: +0.0364 (+3.64 percentage points)
Saved trial results to:
C:\Users\danie\Documents\CSE242\repo_assignments\Assignment3\results\noise_comparison.csv

trial	baseline_accuracy	knn_accuracy
1	0.8345	0.8615
2	0.833	0.872
3	0.844	0.8675
4	0.8445	0.87
5	0.8395	0.8725
6	0.841	0.8715
7	0.8405	0.869
8	0.793	0.848
9	0.824	0.8615
10	0.8425	0.8715
11	0.822	0.86
12	0.819	0.8645
13	0.847	0.8645
14	0.8415	0.872
15	0.816	0.8715
16	0.8285	0.8485
17	0.8235	0.8695
18	0.843	0.87
19	0.784	0.8335
20	0.798	0.869

AI Note

Being a DRC student, I used AI to help me understand the assignment, debugging and helping write the report nicely from my results and my notes.